🚨 Problem Statement 1:"PROMPT POLICE: Build an AI Bouncer That Catches Jailbreaks in Real Time"

💥 Can you catch a malicious prompt even when it's hiding in plain sight?

You know how AI chatbots suddenly go all "sorry, can't help with that" the moment you ask something even slightly sus? Yeah… that's the safety kicking in. But what if someone *doesn't* ask directly? What if they sneak the same request inside a fantasy story, hide it in Base64, or bury it at the end of a 2,000-word rant about sourdough bread?

Welcome to the world of **jailbreaks** — where prompts don't break the rules… they *bend them*.

From roleplay tricks and fake "developer mode" to encoding hacks and multi-turn manipulation, attackers are getting insanely creative — and honestly, they're winning more often than companies would like to admit. And the scary part? —New tricks are dropping every week.

👉 **Because in the real world, attackers don't play fair — and your model shouldn't be easy to fool.**

🎯 **Your Mission:** Build a binary classifier pipeline that takes in a raw user prompt and outputs: SAFE or ADVERSARIAL, with a confidence score between 0 and 1.

Technical Requirements (pick your depth):

The baseline approach is to use TF-IDF or bag-of-words features fed into a logistic regression or random forest. Get this working first, a clean pipeline that preprocesses text, extracts features, trains, and evaluates. The intermediate approach involves using sentence embeddings (via sentence-transformers models like all-MiniLM-L6-v2 are small and fast) to convert prompts into dense vectors, then classifying them with a simple neural network or XGBoost. This captures semantic intent, not just keywords, which is exactly how production-grade systems like Bayora work.
The ambitious stretch involves adding a multi-signal detector: combine your embedding classifier with rule-based checks for Base64 encoding detection, prompt length anomaly flagging, and persona-override keyword patterns. Ensemble the signals. This mirrors real-world adversarial detection pipelines where no single method catches everything.

Evaluation Metric: You'll be judged on F1 Score (because accuracy is misleading with imbalanced classes) and False Positive Rate. Here's why FPR matters: if your system blocks 5% of legitimate users, that's thousands of frustrated people per day. The target is under 1% FPR with over 90% true positive rate. Welcome to the real tradeoff every AI safety engineer faces.

Why This Matters: Every company deploying an LLM right now — from banks to hospitals to food delivery apps is one clever prompt away from a PR disaster. Samsung lost trade secrets. Air Canada got sued over a chatbot's made-up policy. A lawyer got sanctioned for citing hallucinated cases. The industry needs detection systems, and there aren't enough people building them.

Dataset: Use the JailbreakBench dataset on HuggingFace (huggingface.co/datasets/JailbreakBench/JBB-Behaviors) for adversarial prompts. Supplement with the OpenAssistant Conversations dataset (huggingface.co/datasets/OpenAssistant/oasst1) for benign user prompts to create a balanced training set.

Starter Toolkit: Python, scikit-learn, sentence-transformers, HuggingFace datasets library. Everything runs on Google Colab free tier.

🤖 Problem Statement 2: "HALLUCINATION HUNTER: Build a Lie Detector for AI-Generated Text"

💥 Can you tell when an AI is confidently making things up?

Here's a fun experiment. Ask any LLM:*"What's the airspeed velocity of a European swallow carrying a coconut according to the 2019 Cambridge Ornithology Review?"*
It will give you a confident, beautifully written answer.
The paper doesn't exist.
The data is invented.
The citations are fabricated. And it will say all of this with the confidence of a tenured professor.

This is called **hallucination** — when an AI generates information that sounds right but is factually wrong, unsupported, or completely made up. It's arguably the most dangerous failure mode in AI today, because the outputs *look* trustworthy.

👉 **Because in the real world, sounding right isn't enough — being right is what matters.**

🎯 **Your Mission:** Build a system that takes two inputs, a source passage (the ground truth the AI was supposed to use) and the AI-generated response and outputs a verdict: FAITHFUL or HALLUCINATED, along with a confidence score.

Technical Requirements (pick your depth):

The baseline approach involves computing semantic similarity between the source and the response using cosine similarity on sentence embeddings. If the response contains claims that are semantically distant from anything in the source passage, flag them. This is crude but surprisingly effective as a first pass. The intermediate approach involves breaking the AI response into individual claims (sentence-level), then for each claim, performing natural language inference (NLI) against the source passage. Use a pre-trained NLI model (like cross-encoder/nli-deberta-v3-small from HuggingFace) to classify each claim as entailed, contradicted, or neutral. Claims classified as contradicted or neutral are hallucination candidates. This is close to how research-grade hallucination detectors actually work. The ambitious stretch involves building a claim extraction + verification pipeline: use a small LLM (or regex + spaCy) to extract atomic factual claims from the response, embed each claim and each source sentence separately, compute an alignment matrix, and flag unaligned claims. Visualize the results so a human reviewer can see exactly where the AI went off-script.

Evaluation Metric: Balanced Accuracy across faithful and hallucinated examples, plus a claim-level precision score if you attempt the stretch goal. We want systems that catch real hallucinations without crying wolf on faithful responses.

Why This Matters: RAG pipelines, where an LLM retrieves documents and then generates answers grounded in them are becoming the default architecture for enterprise AI. But "grounded" is aspirational, not guaranteed. The LLM can still ignore, distort, or fabricate beyond what the retrieved documents actually say. Companies deploying RAG need automated verification that the output is actually faithful to the source.

Dataset: Use the HaluEval benchmark (github.com/RUCAIBox/HaluEval) — it provides thousands of LLM-generated QA pairs, summarization outputs, and dialogue responses, each labeled as hallucinated or faithful, with the original source passages included. It's purpose-built for this exact task. For additional challenge, try the TRUE benchmark (github.com/google-research/true) which covers factual consistency across summarization datasets.

Starter Toolkit: Python, sentence-transformers, HuggingFace transformers (for NLI models), spaCy (for claim extraction), Colab-friendly.